

Session 7 : Textures — Les Fondamentaux

💡 Objectifs de la session

Comprendre ce qu'est une texture, comment elle est appliquée sur une géométrie via les UVs, comment le GPU la filtre et l'échantillonne, et découvrir le **set de textures PBR** qui définit l'apparence d'un matériau moderne. La session suit le pipeline réel : **créer et préparer les textures dans Blender**, puis **les intégrer dans un moteur** (Babylon.js / Node Material Editor).

💡 Rappel : les UVs vus en Session 1

En Session 1, nous avons défini les **UVs** — des coordonnées $(u, v) \in [0, 1]^2$ qui relient chaque sommet à un pixel d'une texture 2D. Cette session approfondit **comment** cette application fonctionne, ce qui peut mal se passer, et comment les moteurs modernes utilisent non pas une mais **plusieurs** textures pour définir un matériau.

Partie 1 : Qu'est-ce qu'une Texture ?

Définition

Au départ, une texture est un fichier image (PNG, JPG, etc.) qui contient des informations visuelles. On va ensuite charger cette image dans la mémoire du GPU (VRAM) pour qu'elle puisse être utilisée par le fragment shader via un **sampler**. Le shader donne au sampler des coordonnées UV, le sampler retourne la couleur du **texel** correspondant — le pixel de la texture.

💡 Texels vs Pixels

Un **texel** est un pixel de la texture. Un **pixel** est un pixel de l'écran. Ce ne sont pas les mêmes ! Une texture 2048×2048 a 4 millions de texels, mais un fragment shader peut en échantillonner beaucoup plus ou beaucoup moins selon la distance à la caméra. C'est ce déséquilibre qui rend le **filtrage** nécessaire (Partie 3).

Le sampler

En GLSL, on déclare une texture comme un uniform `sampler2D` et on lit une couleur avec `texture(sampler, uv)` :

```
uniform sampler2D uAlbedoMap;    // la texture
in vec2 vUV;                    // UV interpolée par le rasterizer

void main() {
    vec3 albedo = texture(uAlbedoMap, vUV).rgb;
    fragColor = vec4(albedo, 1.0);
}
```

Le sampler est l'interface entre le shader et la VRAM — il gère le filtrage, le wrapping et les mipmaps automatiquement (voir Parties 3 et 4).

Partie 2 : UV Mapping — Le Dépliage dans Blender

Le problème du dépliage

Une texture 2D est plate. Un mesh 3D ne l'est pas. Le **UV mapping** consiste à “déplier” la surface 3D sur le plan 2D de la texture — comme déplier une boîte en carton pour la poser à plat. Chaque sommet reçoit une coordonnée (u, v) qui pointe dans la texture.

1. Seams, islands et stretching

Vocabulaire du dépliage

- **Seam** (couture) : une arête où l'on “coupe” le mesh pour pouvoir le déplier. Comme couper un papier cadeau.
- **Island** (îlot) : un groupe de faces connectées, dépliées ensemble. Un mesh déplié produit plusieurs îlots dans la texture.
- **Stretching** : quand une face 3D est projetée sur une zone 2D trop petite ou disproportionnée — la texture apparaît étirée ou compressée.
- **Packing** : disposition des îlots dans la texture pour minimiser l'espace gaspillé. Les îlots ne doivent pas se chevaucher (sauf si volontaire).

💡 Démo Blender — dépliage d'un cube

Dans Blender : sélectionner le mesh → Mode Édition → U → **Smart UV Project** ou **Unwrap**. Observer dans l'UV Editor :

- Les seams automatiques créent des îlots.
- Chaque îlot est un groupe de faces dépliées.
- Si on déplace un îlot hors de la zone $[0, 1]^2$, la texture ne s'applique plus correctement (voir Partie 4 — wrapping).

2. Interpolation des UVs par le rasterizer

Comment les UVs arrivent au fragment shader

Les UVs sont des **attributs par sommet** (définis en Session 1). Le vertex shader les passe au fragment shader via un varying (ou out/in). Le rasterizer **interpole** ces UVs à travers chaque triangle — chaque fragment reçoit une UV interpolée, pas l'UV d'un sommet.

$$UV_{\text{fragment}} = \alpha \cdot UV_A + \beta \cdot UV_B + \gamma \cdot UV_C$$

où α, β, γ sont les coordonnées barycentriques du fragment dans le triangle. C'est pourquoi une texture appliquée sur un grand triangle apparaît **continue** — chaque pixel lit un texel différent, interpolé à partir des trois sommets.

Partie 3 : Le Filtrage

💡 Le problème central

Quand une texture est affichée à l'écran, il y a deux cas extrêmes :

- **Minification** : la texture est plus grande que l'écran (un mur de briques vu de loin — plusieurs texels par pixel).

- **Magnification** : la texture est plus petite que l'écran (un mur vu de très près — plusieurs pixels par texel).

Le **filtrage** détermine comment le sampler choisit la couleur du texel dans ces deux cas. Un mauvais filtrage produit des artefacts visibles : crénelage (aliasing), flou, ou scintillement.

3. Magnification — Nearest vs Bilinear

Nearest neighbor

Le sampler prend le texel le plus proche des UVs. Résultat : blocs nets, look “pixel art”. Rapide mais crénelé.

```
// Équivalent conceptuel
vec2 texelCoord = uv * textureSize;
ivec2 nearest = ivec2(floor(texelCoord + 0.5));
vec3 color = texelFetch(sampler, nearest, 0).rgb;
```

Bilinear filtering

Le sampler prend les 4 texels les plus proches et interpole linéairement entre eux. Résultat : transitions douces, pas de blocs. C'est le filtrage par défaut.

$$\text{color} = \text{lerp}(\text{lerp}(t_{00}, t_{10}, f_x), \text{lerp}(t_{01}, t_{11}, f_x), f_y)$$

où f_x, f_y sont les fractions de position dans la cellule de texels.

4. Minification et Mipmaps

Le problème de la minification

Quand un mur de briques est vu de loin, chaque pixel couvre **des dizaines** de texels. Si le sampler ne lit qu'un seul texel (nearest ou bilinear), il **rate** la plupart des briques — on obtient du **moiré** (scintillement, motifs parasites). C'est l'aliasing de texture.

Mipmaps — la solution

Le GPU génère automatiquement une **pyramide** de versions de la texture de plus en plus petites : 2048→1024→512→256→128→64→32→16→8→4→2→1. Chaque niveau est un quart de la résolution du précédent (moitié en largeur et hauteur).

Le sampler choisit le niveau de mipmap approprié selon la distance à la caméra : de loin, il lit un texel d'une petite mipmap (qui représente déjà la moyenne de plusieurs texels). Plus de moiré.

- **Trilinear filtering** : bilinear sur les deux niveaux de mipmap les plus proches, puis interpolation entre les deux. Transitions douces entre les niveaux.
- **Coût mémoire** : les mipmaps ajoutent $\sim 33\%$ de mémoire (somme d'une série géométrique : $1 + \frac{1}{4} + \frac{1}{16} + \dots \approx \frac{4}{3}$).

💡 Dans Babylon.js / Godot

Les mipmaps sont activés par défaut sur les textures 2D. On peut les désactiver pour des textures d'interface (UI) ou des textures pixel art où l'on **veut** l'effet bloc.

```
// Babylon.js
const tex = new BABYLON.Texture("textures/bricks.jpg", scene);
tex.wrapU = BABYLON.Texture.WRAP_ADDRESSMODE; // voir Partie 4
// Les mipmaps sont générés automatiquement.
```

Partie 4 : Le Wrapping

Que se passe-t-il hors de [0,1] ?

Les UVs peuvent dépasser l'intervalle [0, 1] — par exemple, un sol qui répète une texture de carrelage sur 10×10 mètres avec des UVs allant de 0 à 10. Le mode de **wrapping** détermine ce que fait le sampler :

- **Repeat** : la texture se répète (carrelage, tiling). Le mode le plus utilisé pour les sols, murs, etc.
- **Clamp** : les UVs hors [0, 1] sont écrêtées sur les bords. La texture s'étire sur les bords. Utilisé pour les textures uniques (un personnage, un logo).
- **Mirror** : la texture se reflète à chaque répétition. Utile pour cacher les seams de tiling.

Wrapping en Babylon.js.

```
const floorTex = new BABYLON.Texture("textures/tiles.jpg", scene);
floorTex.wrapU = BABYLON.Texture.WRAP_ADDRESSMODE; // repeat
floorTex.wrapV = BABYLON.Texture.WRAP_ADDRESSMODE;
floorTex.uScale = 10; // répéter 10 fois en U
floorTex.vScale = 10; // répéter 10 fois en V

// Pour une texture unique (personnage) :
const skinTex = new BABYLON.Texture("textures/skin.jpg", scene);
skinTex.wrapU = BABYLON.Texture.CLAMP_ADDRESSMODE;
skinTex.wrapV = BABYLON.Texture.CLAMP_ADDRESSMODE;
```

Partie 5 : Les Formats de Texture

Deux grandes catégories

- **Formats non compressés GPU** : PNG, JPEG, TIFF. Le GPU doit les **décompresser** à l'envoi, puis stocke les pixels bruts en VRAM. Simple mais gourmand en mémoire : une texture 2048×2048 RGBA = $2048 \times 2048 \times 4 = 16$ "Mo".
- **Formats compressés GPU** : DDS (DirectX), KTX (Khronos), avec des algorithmes de compression **bloc** comme BCn (Desktop), ETC (Mobile), ASTC (Mobile moderne). Le GPU lit les blocs compressés **directement** et les décompresse à la volée dans le shader. Ratio de compression $\sim 4 : 1$ à $\sim 8 : 1$ sans coût de décompression.

💡 Quand utiliser quoi ?

- **Prototypage / démo** : PNG ou JPEG — simple, universel.
- **Production desktop** : BCn (via DDS ou KTX). BC1 pour l'albedo (1 bit alpha), BC5 pour les normal maps (2 canaux, haute précision), BC7 pour les textures avec alpha.
- **Production mobile** : ASTC (qualité réglable, ratio excellent).
- **HDR / lightmaps** : EXR ou HDR (valeurs > 1.0, vu en Session 6).

Partie 6 : Le Set de Textures PBR

! Une texture ne suffit pas

En Session 5, on a vu que `PBRMaterial` de Babylon.js utilise `albedoColor`, `metallic`, `roughness` — des valeurs **uniformes** (une couleur pour tout l'objet). Mais un vrai matériau n'est pas uniforme : une planche de bois a des fibres, des nœuds, des variations de brillance. C'est pourquoi on utilise **plusieurs textures** — un **set PBR** — pour contrôler chaque propriété **par texel**.

Les maps essentielles d'un set PBR

Map	Canaux	Rôle
Albedo / Base Color	RGB	La couleur de base du matériau, sans aucune information d'éclairage. C'est ce que l'objet aurait sous une lumière blanche uniforme.
Normal Map	RGB	Perturbe la normale de surface par texel pour simuler du relief sans géométrie. Détail en Partie 7.
Roughness	1 (grayscale)	Contrôle la rugosité de la surface : 0 = miroir parfait (spéculaire net), 1 = mate (spéculaire diffus). Varie par texel : une planche usée est plus rugueuse que sa vernie.
Metallic	1 (grayscale)	Indique si le texel est un métal (1.0) ou un diélectrique (0.0). Les métaux réfléchissent teintés par leur albedo ; les diélectriques réfléchissent blanc. Généralement binaire, parfois lissé.
Ambient Occlusion	1 (grayscale)	Assombrit les crevasses et zones occluses. Vu en Session 6 — souvent baked depuis la géométrie high-poly.
Height / Displacement	1 (grayscale)	Carte de hauteur pour le parallax mapping (Session 8) ou le déplacement réel (déplacement des sommets).

Table 1: Un set PBR complet : 6 maps qui contrôlent chaque propriété du matériau indépendamment, par texel.

💡 Démo Blender — le set de briques

Dans Blender, ouvrir `Blender/assets/textures/StoneBricksSplitface001/`. Le set contient :

- `StoneBricksSplitface001_COL_2K.jpg` — Albedo
- `StoneBricksSplitface001_NRM_2K.jpg` — Normal
- `StoneBricksSplitface001_REFL_2K.jpg` — Réflectivité (inverse de roughness)
- `StoneBricksSplitface001_GLOSS_2K.jpg` — Glossiness
- `StoneBricksSplitface001_AO_2K.jpg` — Ambient Occlusion
- `StoneBricksSplitface001_DISP_2K.jpg` — Displacement

Dans le **Shader Editor** de Blender, brancher chaque map dans le noeud **Principled BSDF** correspondant. Observer comment chaque map affecte le rendu indépendamment.

Partie 7 : Les Normal Maps en Détail

Le principe

Une normal map est une texture RGB où chaque texel stocke un **vecteur normal perturbé** (n_x, n_y, n_z) . Au lieu d'utiliser la normale géométrique de la surface (interpolée depuis les sommets), le fragment shader **lit** la normale dans la texture et l'utilise pour le calcul d'éclairage.

Résultat : les fibres d'une planche, les écailles d'un reptile, les briques d'un mur semblent en **relief** — sans ajouter un seul triangle. C'est l'illusion la plus rentable en infographie.

❗ Pourquoi les normal maps sont bleues ?

Les normales sont stockées dans l'**espace tangent** — un repère local à chaque surface, défini par la tangente, la binormale et la normale géométrique (vus en Session 1). Dans ce repère :

- n_x (rouge) : déviation gauche/droite
- n_y (vert) : déviation avant/arrière
- n_z (bleu) : profondeur (normale "vers l'extérieur")

Une surface **plate** a une normale $(0, 0, 1)$ — donc $R = 0, G = 0, B = 1$ — **bleu**. Les normal maps sont donc dominées par des tons bleus. Une zone rouge penche à droite, une zone verte penche vers l'avant.

Normal mapping dans un fragment shader (GLSL).

```
uniform sampler2D uNormalMap;
in vec2 vUV;
in vec3 vNormal;    // normale géométrique (interpolée)
in vec3 vTangent;   // tangente (interpolée)
in vec3 vBitangent; // binormale (interpolée)

void main() {
    // 1. Lire la normale perturbée dans la texture ( [0,1] -> [-1,1] )
    vec3 n = texture(uNormalMap, vUV).rgb * 2.0 - 1.0;

    // 2. Construire la matrice TBN (tangent -> world space)
    mat3 TBN = mat3(normalize(vTangent), normalize(vBitangent), normalize(vNormal));

    // 3. Transformer la normale de l'espace tangent vers l'espace monde
    vec3 worldNormal = normalize(TBN * n);

    // 4. Utiliser worldNormal pour l'éclairage (Lambert, etc.)
    float diff = max(dot(worldNormal, lightDir), 0.0);
    fragColor = vec4(albedo * diff, 1.0);
}
```

La matrice **TBN** (Tangent, Bitangent, Normal) convertit du repère local de la surface vers le repère monde — c'est le pont entre la normale stockée dans la texture et la normale utilisée pour l'éclairage.

Partie 8 : Bump, Normal et Displacement — Les Trois Approches du Relief

! Le problème commun

Une surface 3D a une géométrie **limitée** — un mur de briques modélisé avec 10 triangles est plat, même si les vraies briques ont des joints, des bosses, des éclats. On veut ajouter du relief **sans** ajouter de polygones. Il existe trois techniques, du moins coûteux au plus réaliste, qui font toutes la même chose — **simuler du relief** — mais avec des compromis différents.

1. Bump Map — l’ancêtre

Principe

Une **bump map** est une texture **grayscale** (niveaux de gris) où chaque texel stocke une **hauteur** relative : blanc = haut, noir = bas. Le fragment shader calcule une normale perturbée à **partir de cette hauteur** en estimant les dérivées partielles :

$$n_x \approx \frac{h(u + \varepsilon, v) - h(u - \varepsilon, v)}{2\varepsilon}$$
$$n_y \approx \frac{h(u, v + \varepsilon) - h(u, v - \varepsilon)}{2\varepsilon}$$
$$n_z = 1$$

(normalisé ensuite)

En pratique, le shader lit 3 ou 4 texels voisins dans la bump map et calcule le **gradient** — la pente locale. Cette pente devient la normale perturbée.

💡 Bump vs Normal — quelle différence ?

Une **bump map** et une **normal map** produisent **exactement le même effet** : une normale perturbée dans le fragment shader. La différence est le **format** :

- **Bump map** : grayscale (1 canal), hauteur. Le shader calcule la normale à la volée.
- **Normal map** : RGB (3 canaux), normale pré-calculée. Le shader lit directement la normale.

Les normal maps ont **remplacé** les bump maps dans les pipelines modernes : la normale est pré-calculée (par Blender, Substance Painter, etc.) avec plus de précision qu’un gradient estimé dans le shader, et stockée dans 3 canaux au lieu d’1. Mais le résultat visuel est le même — l’une est juste plus précise que l’autre.

En résumé : une normal map est une bump map **pré-calculée et plus précise**. On n’utilise presque plus les bump maps aujourd’hui.

2. Displacement Map — le vrai relief

Principe

Une **displacement map** (ou **height map**) est aussi une texture grayscale de hauteur — **comme une bump map**. Mais au lieu de truquer la normale dans le **fragment shader**, on déplace **réellement les sommets** de la géométrie dans le **vertex shader** :

$$\text{position}_{\text{déplacée}} = \text{position}_{\text{originale}} + \text{normale} \cdot h(\text{UV}) \cdot k$$

où $h(UV)$ est la hauteur lue dans la texture et k un facteur d'intensité. La surface est **physiquement** déformée — le relief est vrai, visible de profil, et la silhouette est correcte.

🗨 La différence fondamentale : vertex vs fragment shader

- **Bump / Normal map** : agissent dans le **fragment shader**. La géométrie est intacte — seules les normales utilisées pour l'éclairage changent. La surface reste plate en réalité.
- **Displacement map** : agit dans le **vertex shader**. La géométrie est **réellement modifiée** — les sommets bougent. La surface est vraiment déformée.

C'est pourquoi le déplacement nécessite **beaucoup** de polygones : si un triangle est grand, le vertex shader ne peut le déplacer qu'à ses 3 sommets — le centre du triangle reste plat (interpolation linéaire). Pour un relief fin, il faut **subdiviser** le mesh en milliers de petits triangles.

Displacement dans un vertex shader (GLSL).

```
uniform sampler2D uHeightMap;
uniform float uDisplacementScale;
in vec3 position;    // position du sommet (object space)
in vec3 normal;      // normale du sommet
in vec2 uv;          // UV du sommet

void main() {
    // 1. Lire la hauteur dans la texture
    float h = texture(uHeightMap, uv).r;    // [0,1]

    // 2. Déplacer le sommet le long de sa normale
    vec3 displaced = position + normal * (h * uDisplacementScale);

    // 3. Transformer en clip space comme d'habitude
    gl_Position = projection * view * model * vec4(displaced, 1.0);
}
```

Le vertex shader lit la height map **par sommet** — pas par fragment. C'est beaucoup moins de lectures de texture (un mesh a des milliers de sommets, mais des millions de fragments), mais chaque sommet ne peut être déplacé qu'une fois.

⚠ Le problème de la résolution géométrique

Si le mesh a trop peu de polygones, le déplacement est **invisible** ou **imprécis** :

- Un plan de 2 triangles déplacé par une height map de briques ne montrera que 2 sommets déplacés — le reste est interpolé. Le relief est inexistant.
- Solution : **subdiviser** le mesh avant le déplacement. Les moteurs modernes font ça dynamiquement avec la **tessellation** (GPU) — le GPU subdivise chaque triangle en plus petits triangles à la volée, puis le vertex shader les déplace.

La tessellation est avancée et hors programme, mais le concept est important : **le déplacement n'a de sens que si la géométrie est assez dense.**

3. Comparaison des trois techniques

Critère	Bump Map	Normal Map	Displacement Map
Shader	Fragment	Fragment	Vertex
Format	Grayscale (1 ch.)	RGB (3 ch.)	Grayscale (1 ch.)
Géométrie modifiée ?	Non	Non	Oui
Silhouette correcte ?	Non	Non	Oui
Coût GPU	Faible	Faible	Élevé (subdivision) ^c
Polygones requis	Quelconque	Quelconque	Beaucoup
Précision du relief	Moyenne (gradient estimé)	Haute (normale pré-calculée)	Parfaite (vrai relief)
Usage aujourd'hui	Obsolète	Standard	Cinématique / tessellation ^d

Table 2: Les trois techniques de relief. La normal map est le standard temps réel — le meilleur compromis qualité/coût. Le displacement est réservé aux cas où la silhouette doit être correcte (cinématique, rendu offline) ou avec la tessellation GPU.

💡 Dans Blender — les trois dans le Shader Editor

Dans le **Shader Editor** de Blender, le noeud **Principled BSDF** propose trois entrées :

- **Normal** : brancher une **Image Texture** + **Normal Map** node (normal map standard).
- **Bump** : brancher une **Image Texture** + **Bump** node (bump map — Blender convertit la hauteur en normale).
- **Displacement** : brancher une **Image Texture** + **Displacement** node. Nécessite que le mesh soit **subdivisé** (modifier **Subdivision Surface**) et que le matériel soit en mode **Displacement and Bump** (pas **Bump only**).

Observer : avec **Displacement**, la silhouette du mesh change réellement — les bords sont dentelés. Avec **Normal** seul, les bords restent droits.

💡 Et le parallax mapping ? (Session 8)

Il existe une **quatrième** technique : le **parallax mapping**, vu en Session 8. C'est un compromis entre normal map et displacement : la géométrie reste plate, mais le fragment shader **décale les UVs** selon l'angle de vue pour simuler la parallaxe du relief. Moins précis que le displacement, mais beaucoup moins coûteux — et meilleur que la normal map seule pour la perception de profondeur.

Partie 9 : Export depuis Blender

Le format GLB — textures embarquées

Pour transférer un modèle texturé de Blender vers un moteur web (Babylon.js, Three.js), on exporte en **GLB** (GLTF binaire) — un format qui embarque la géométrie, les UVs, les textures et les matériaux dans un seul fichier.

Dans Blender : **File** → **Export** → **glTF 2.0 (.glb)**. Les options importantes :

- **Format** : glb (binaire, un fichier) ou gltf + textures séparées.
- **Materials** : cocher **Export materials** pour embarquer les textures PBR.
- **UVs** : coché par défaut — sans UVs, les textures ne s'appliquent pas.

🔗 Vérifier l'export

Avant d'importer dans le moteur, vérifier que le GLB contient bien les textures :

- Ouvrir le GLB dans un viewer (ex: gltf-viewer) — les textures PBR doivent s'afficher.
- Si le modèle apparaît gris, les textures n'ont pas été embarquées — revoir les options d'export.

Partie 10 : TP — Set PBR Complet dans Babylon.js

🔗 Objectif du TP

Importer un modèle texturé (exporté de Blender ou téléchargé) dans Babylon.js, et reproduire le branchement du set PBR dans le **Node Material Editor** — comme on l'a vu dans le Shader Editor de Blender, mais côté moteur.

Mise en place

- Télécharger un set PBR sur ambientCG (gratuit, CC0) — par exemple **StoneBricks** ou **WoodPlanks**.

Vous pouvez aussi utiliser les assets fournis dans le cours.

- Ouvrir le **Node Material Editor** de Babylon.js : nme.babylonjs.com.
- Créer un material avec un **PBR Metallic Roughness** block.
- Vous pouvez regarder un setup ici: [PBR setup example](#)

Missions

Mission 1 — Albedo

Ajouter un **Texture** block, charger la map `_COL_2K.jpg`, la brancher sur l'entrée albedo du PBR block. Observer la couleur de base.

Mission 2 — Normal Map

Ajouter un second **Texture** block pour `_NRM_2K.jpg`, la brancher sur `normal`. Observer le relief apparent — les briques semblent en 3D. Tourner la caméra de profil pour voir l'illusion se briser.

Mission 3 — Roughness

Brancher la map `_GLOSS_2K.jpg` (ou `_ROUGH_2K.jpg` selon le set) sur roughness. Observer : les zones vernies sont brillantes (spéculaire net), les zones poreuses sont mates.

Mission 4 — Ambient Occlusion

Brancher `_AO_2K.jpg` sur l'entrée `ambient occlusion` (ou la multiplier avec l'albedo dans un **Multiply** block). Observer comment les crevasses s'assombrissent.

Mission 5 — Comparer

Désactiver chaque map une par une et observer la dégradation. Sans normal map : la surface paraît plate. Sans roughness : tout est uniformément brillant. Sans AO : les crevasses manquent de profondeur.